

Unity Coursework Code Submission (C# Source)

Volleyball exercise + Tennis exercise - compiled in text format with provenance markers

Each file is reproduced verbatim as text, with explicit start and end comment markers indicating sections claimed as personally written without assistance

Volleyball exercise

Volleyball - FollowCamera.cs

Provenance header

- Project: Volleyball
- File: FollowCamera.cs
- Authorship status: Written by me without assistance.

Code (text format):

```
// ===== BEGIN SECTION =====
using UnityEngine;

public class FollowCamera : MonoBehaviour
{
    public Transform target;           // Player
    public Vector3 offset = new Vector3(0f, 6f, -10f);
    public float followSmooth = 8f;
    public float rotateSmooth = 10f;

    void LateUpdate()
    {
        if (target == null) return;

        // Smooth position follow
        Vector3 desiredPos = target.position + offset;
        transform.position = Vector3.Lerp(transform.position, desiredPos, followSmooth * Time.deltaTime);

        // Smooth look at target
        Quaternion desiredRot = Quaternion.LookRotation(target.position - transform.position, Vector3.up);
        transform.rotation = Quaternion.Slerp(transform.rotation, desiredRot, rotateSmooth * Time.deltaTime);
    }
}
// ===== END SECTION =====
```

Volleyball - PlayerController.cs

Provenance header

- Project: Volleyball
- File: PlayerController.cs
- Authorship status: Written by me without assistance.

Code (text format):

```
// ===== BEGIN SECTION =====
using UnityEngine;

[RequireComponent(typeof(Rigidbody))]
public class PlayerController : MonoBehaviour
{
    [Header("Movement")]
    public float moveSpeed = 6f;
    public float acceleration = 25f;

    [Header("Jump")]
    public float jumpForce = 7f;
    public Transform groundCheck; // empty object at feet
    public float groundCheckRadius = 0.25f;
    public LayerMask groundMask; // set to Ground layer

    [Header("References")]
    public Transform cameraTransform;

    private Rigidbody rb;
    private Vector3 moveInput;

    void Awake()
    {
        rb = GetComponent<Rigidbody>();

        if (cameraTransform == null && Camera.main != null)
            cameraTransform = Camera.main.transform;
    }

    void Update()
    {
        // WASD input
        float h = Input.GetAxisRaw("Horizontal");
        float v = Input.GetAxisRaw("Vertical");
        moveInput = new Vector3(h, 0f, v).normalized;

        // Jump
        if (Input.GetButtonDown("Jump") && IsGrounded())
        {
            rb.AddForce(Vector3.up * jumpForce, ForceMode.Impulse);
        }
    }

    void FixedUpdate()
    {
        if (cameraTransform == null) return;

        // Move relative to camera facing (ignoring camera pitch)
        Vector3 camForward = cameraTransform.forward;
        camForward.y = 0f;
```

```

camForward.Normalize();

Vector3 camRight = cameraTransform.right;
camRight.y = 0f;
camRight.Normalize();

Vector3 desiredDir = (camForward * moveInput.z + camRight * moveInput.x).normalized;
Vector3 desiredVelocity = desiredDir * moveSpeed;

// Keep existing Y velocity (gravity/jump), drive XZ toward desired
Vector3 currentVel = rb.linearVelocity;
Vector3 targetVel = new Vector3(desiredVelocity.x, currentVel.y, desiredVelocity.z);

rb.linearVelocity = Vector3.MoveTowards(currentVel, targetVel, acceleration * Time.fixedDeltaTime);
}

bool IsGrounded()
{
    if (groundCheck == null) return false;
    return Physics.CheckSphere(groundCheck.position, groundCheckRadius, groundMask, QueryTriggerInteraction.Ignore);
}

// Optional: visualize ground check
void OnDrawGizmosSelected()
{
    if (groundCheck == null) return;
    Gizmos.color = Color.yellow;
    Gizmos.DrawWireSphere(groundCheck.position, groundCheckRadius);
}
}
// ===== END SECTION =====

```

Volleyball - PlayerMovement.cs

Provenance header

- Project: Volleyball
- File: PlayerMovement.cs
- Authorship status: Written by me without assistance.

Code (text format):

```
// ===== BEGIN SECTION =====
using UnityEngine;

[RequireComponent(typeof(Rigidbody))]
public class PlayerMovement : MonoBehaviour
{
    public float moveSpeed = 6f;

    private Rigidbody rb;
    private Vector3 input;

    void Awake()
    {
        rb = GetComponent<Rigidbody>();
    }

    void Update()
    {
        // WASD input
        float h = Input.GetAxisRaw("Horizontal"); // A/D
        float v = Input.GetAxisRaw("Vertical"); // W/S

        input = new Vector3(h, 0f, v).normalized;
    }

    void FixedUpdate()
    {
        // Move in world space (simple). If you want relative-to-camera, tell me.
        Vector3 targetVelocity = input * moveSpeed;
        Vector3 velocityChange = targetVelocity - new Vector3(rb.linearVelocity.x, 0f, rb.linearVelocity.z);

        rb.AddForce(velocityChange, ForceMode.VelocityChange);
    }
}
// ===== END SECTION =====
```

Volleyball - ThirdPersonOrbitCamera.cs

Provenance header

- Project: Volleyball
- File: ThirdPersonOrbitCamera.cs
- Authorship status: Written by me without assistance.

Code (text format):

```
// ===== BEGIN SECTION =====
using UnityEngine;

public class ThirdPersonOrbitCamera : MonoBehaviour
{
    public Transform target;           // Player
    public Vector3 targetOffset = new Vector3(0f, 1.5f, 0f);

    [Header("Distance")]
    public float distance = 6f;
    public float minDistance = 2f;
    public float maxDistance = 10f;

    [Header("Mouse Look")]
    public float sensitivityX = 3f;
    public float sensitivityY = 2f;
    public float minPitch = -20f;
    public float maxPitch = 60f;

    [Header("Smoothing")]
    public float positionSmooth = 12f;

    [Header("Optional: Camera Collision")]
    public bool enableCollision = true;
    public LayerMask collisionMask;    // set to Default + Environment, exclude Player
    public float collisionRadius = 0.25f;

    private float yaw;
    private float pitch;

    void Start()
    {
        if (target == null)
        {
            Debug.LogError("ThirdPersonOrbitCamera: Assign a target (Player).");
            enabled = false;
            return;
        }

        Vector3 angles = transform.eulerAngles;
        yaw = angles.y;
        pitch = angles.x;

        // Optional: lock cursor for 3rd person controls
        Cursor.lockState = CursorLockMode.Locked;
        Cursor.visible = false;
    }

    void LateUpdate()
    {
        // Mouse input
```

```

yaw += Input.GetAxis("Mouse X") * sensitivityX;
pitch -= Input.GetAxis("Mouse Y") * sensitivityY;
pitch = Mathf.Clamp(pitch, minPitch, maxPitch);

// Zoom (optional)
float scroll = Input.GetAxis("Mouse ScrollWheel");
if (Mathf.Abs(scroll) > 0.001f)
{
    distance = Mathf.Clamp(distance - scroll * 2f, minDistance, maxDistance);
}

Quaternion rot = Quaternion.Euler(pitch, yaw, 0f);
Vector3 targetPos = target.position + targetOffset;

// Desired camera position
Vector3 desiredPos = targetPos - (rot * Vector3.forward * distance);

// Collision handling: push camera forward if blocked
if (enableCollision)
{
    Vector3 dir = (desiredPos - targetPos).normalized;
    float desiredDist = Vector3.Distance(targetPos, desiredPos);

    if (Physics.SphereCast(targetPos, collisionRadius, dir, out RaycastHit hit, desiredDist, collisionMask,
QueryTriggerInteraction.Ignore))
    {
        desiredPos = targetPos + dir * (hit.distance - 0.05f);
    }
}

// Smooth follow
transform.position = Vector3.Lerp(transform.position, desiredPos, positionSmooth * Time.deltaTime);
transform.rotation = rot;
}
}
// ===== END SECTION =====

```

Volleyball - ReadmeEditor.cs

]

Provenance header

- Project: Volleyball
- File: ReadmeEditor.cs
- Authorship status: EXTERNAL TEMPLATE: Included from Unity project template or sample content.

Code (text format):

```
// ===== BEGIN SECTION =====
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEditor;
using System;
using System.IO;
using System.Reflection;

[CustomEditor(typeof(Readme))]
[InitializeOnLoad]
public class ReadmeEditor : Editor
{
    static string s_ShowedReadmeSessionStateName = "ReadmeEditor.showedReadme";

    static string s_ReadmeSourceDirectory = "Assets/TutorialInfo";

    const float k_Space = 16f;

    static ReadmeEditor()
    {
        EditorApplication.delayCall += SelectReadmeAutomatically;
    }

    static void RemoveTutorial()
    {
        if (EditorUtility.DisplayDialog("Remove Readme Assets",

            $"All contents under {s_ReadmeSourceDirectory} will be removed, are you sure you want to proceed?",
            "Proceed",
            "Cancel"))
        {
            if (Directory.Exists(s_ReadmeSourceDirectory))
            {
                FileUtil.DeleteFileOrDirectory(s_ReadmeSourceDirectory);
                FileUtil.DeleteFileOrDirectory(s_ReadmeSourceDirectory + ".meta");
            }
            else
            {
                Debug.Log($"Could not find the Readme folder at {s_ReadmeSourceDirectory}");
            }

            var readmeAsset = SelectReadme();
            if (readmeAsset != null)
            {
                var path = AssetDatabase.GetAssetPath(readmeAsset);
                FileUtil.DeleteFileOrDirectory(path + ".meta");
                FileUtil.DeleteFileOrDirectory(path);
            }
        }
    }
}
```

```

        AssetDatabase.Refresh();
    }
}

static void SelectReadmeAutomatically()
{
    if (!SessionState.GetBool(s_ShowedReadmeSessionStateName, false))
    {
        var readme = SelectReadme();
        SessionState.SetBool(s_ShowedReadmeSessionStateName, true);

        if (readme && !readme.loadedLayout)
        {
            LoadLayout();
            readme.loadedLayout = true;
        }
    }
}

static void LoadLayout()
{
    var assembly = typeof(EditorApplication).Assembly;
    var windowLayoutType = assembly.GetType("UnityEditor.WindowLayout", true);
    var method = windowLayoutType.GetMethod("LoadWindowLayout", BindingFlags.Public | BindingFlags.Static);
    method.Invoke(null, new object[] { Path.Combine(Application.dataPath, "TutorialInfo/Layout.wlt"), false });
}

static Readme SelectReadme()
{
    var ids = AssetDatabase.FindAssets("Readme t:Readme");
    if (ids.Length == 1)
    {
        var readmeObject = AssetDatabase.LoadMainAssetAtPath(AssetDatabase.GUIDToAssetPath(ids[0]));

        Selection.objects = new UnityEngine.Object[] { readmeObject };

        return (Readme)readmeObject;
    }
    else
    {
        Debug.Log("Couldn't find a readme");
        return null;
    }
}

protected override void OnHeaderGUI()
{
    var readme = (Readme)target;
    Init();

    var iconWidth = Mathf.Min(EditorGUIUtility.currentViewWidth / 3f - 20f, 128f);

    GUILayout.BeginHorizontal("In BigTitle");
    {
        if (readme.icon != null)
        {
            GUILayout.Space(k_Space);
            GUILayout.Label(readme.icon, GUILayout.Width(iconWidth), GUILayout.Height(iconWidth));
        }
        GUILayout.Space(k_Space);
        GUILayout.BeginVertical();
        {
            GUILayout.FlexibleSpace();
            GUILayout.Label(readme.title, TitleStyle);
            GUILayout.FlexibleSpace();
        }
    }
}

```

```

        GUILayout.EndVertical();
        GUILayout.FlexibleSpace();
    }
    GUILayout.EndHorizontal();
}

public override void OnInspectorGUI()
{
    var readme = (Readme)target;
    Init();

    foreach (var section in readme.sections)
    {
        if (!string.IsNullOrEmpty(section.heading))
        {
            GUILayout.Label(section.heading, HeadingStyle);
        }

        if (!string.IsNullOrEmpty(section.text))
        {
            GUILayout.Label(section.text, BodyStyle);
        }

        if (!string.IsNullOrEmpty(section.linkText))
        {
            if (LinkLabel(new GUIContent(section.linkText)))
            {
                Application.OpenURL(section.url);
            }
        }

        GUILayout.Space(k_Space);
    }

    if (GUILayout.Button("Remove Readme Assets", ButtonStyle))
    {
        RemoveTutorial();
    }
}

bool m_Initialized;

GUIStyle LinkStyle
{
    get { return m_LinkStyle; }
}

[SerializeField]
GUIStyle m_LinkStyle;

GUIStyle TitleStyle
{
    get { return m_TitleStyle; }
}

[SerializeField]
GUIStyle m_TitleStyle;

GUIStyle HeadingStyle
{
    get { return m_HeadingStyle; }
}

[SerializeField]
GUIStyle m_HeadingStyle;

GUIStyle BodyStyle
{

```

```

    get { return m_BodyStyle; }
}

[SerializeField]
GUIStyle m_BodyStyle;

GUIStyle ButtonStyle
{
    get { return m_ButtonStyle; }
}

[SerializeField]
GUIStyle m_ButtonStyle;

void Init()
{
    if (m_Initialized)
        return;
    m_BodyStyle = new GUIStyle(EditorStyles.label);
    m_BodyStyle.wordWrap = true;
    m_BodyStyle.fontSize = 14;
    m_BodyStyle.richText = true;

    m_TitleStyle = new GUIStyle(m_BodyStyle);
    m_TitleStyle.fontSize = 26;

    m_HeadingStyle = new GUIStyle(m_BodyStyle);
    m_HeadingStyle.fontStyle = FontStyle.Bold;
    m_HeadingStyle.fontSize = 18;

    m_LinkStyle = new GUIStyle(m_BodyStyle);
    m_LinkStyle.wordWrap = false;

    // Match selection color which works nicely for both light and dark skins
    m_LinkStyle.normal.textColor = new Color(0x00 / 255f, 0x78 / 255f, 0xDA / 255f, 1f);
    m_LinkStyle.stretchWidth = false;

    m_ButtonStyle = new GUIStyle(EditorStyles.miniButton);
    m_ButtonStyle.fontStyle = FontStyle.Bold;

    m_Initialized = true;
}

bool LinkLabel(GUIContent label, params GUILayoutOption[] options)
{
    var position = GUILayoutUtility.GetRect(label, LinkStyle, options);

    Handles.BeginGUI();
    Handles.color = LinkStyle.normal.textColor;
    Handles.DrawLine(new Vector3(position.xMin, position.yMax), new Vector3(position.xMax, position.yMax));
    Handles.color = Color.white;
    Handles.EndGUI();

    EditorGUIUtility.AddCursorRect(position, MouseCursor.Link);

    return GUI.Button(position, label, LinkStyle);
}
}
// ===== END SECTION =====

```

Volleyball - Readme.cs

Provenance header

- Project: Volleyball
- File: Readme.cs
- Authorship status: EXTERNAL TEMPLATE: Included from Unity project template or sample content.

Code (text format):

```
// ===== BEGIN SECTION =====
using System;
using UnityEngine;

public class Readme : ScriptableObject
{
    public Texture2D icon;
    public string title;
    public Section[] sections;
    public bool loadedLayout;

    [Serializable]
    public class Section
    {
        public string heading, text, linkText, url;
    }
}
// ===== END SECTION =====
```

Tennis exercise

Tennis - BallController.cs

Provenance header

- Project: Tennis
- File: BallController.cs
- Authorship status: Written by me without assistance.

Code (text format):

```
// ===== BEGIN SECTION =====
using UnityEngine;

[RequireComponent(typeof(Rigidbody))]
public class BallController : MonoBehaviour
{
    [Header("Movement")]
    [Tooltip("Acceleration force applied while holding WASD.")]
    public float moveForce = 20f;

    [Tooltip("Maximum horizontal speed (XZ). Helps keep control stable.")]
    public float maxSpeed = 8f;

    [Header("Stopping")]
    [Tooltip("How quickly horizontal movement decays when no input is held. Higher = stops faster.")]
    public float stopDamping = 8f;

    [Tooltip("If horizontal speed is below this, snap it to zero to prevent micro-sliding.")]
    public float stopThreshold = 0.05f;

    [Header("Grounding (optional but recommended)")]
    public LayerMask groundMask = ~0; // Everything by default
    public float groundCheckDistance = 0.6f;

    private Rigidbody _rb;

    private void Awake()
    {
        _rb = GetComponent<Rigidbody>();
    }

    private void FixedUpdate()
    {
        float h = Input.GetAxisRaw("Horizontal"); // A/D
        float v = Input.GetAxisRaw("Vertical"); // W/S

        Vector3 input = new Vector3(h, 0f, v);
        if (input.sqrMagnitude > 1f) input.Normalize();

        bool grounded = IsGrounded();

        if (grounded)
        {
            if (input.sqrMagnitude > 0.001f)
            {
                // Move with force while input is held
                Vector3 force = input * moveForce;
                _rb.AddForce(force, ForceMode.Acceleration);
            }
        }
    }
}
```

```

    }
    else
    {
        // No input: damp horizontal velocity so the ball settles
        ApplyStopDamping();
    }
}

LimitHorizontalSpeed();
}

private bool IsGrounded()
{
    return Physics.Raycast(transform.position, Vector3.down, groundCheckDistance, groundMask);
}

private void ApplyStopDamping()
{
    Vector3 vel = _rb.linearVelocity;
    Vector3 horizontal = new Vector3(vel.x, 0f, vel.z);

    // Snap tiny drift to zero
    if (horizontal.magnitude < stopThreshold)
    {
        _rb.linearVelocity = new Vector3(0f, vel.y, 0f);
        return;
    }

    // Smoothly reduce horizontal velocity toward zero
    Vector3 damped = Vector3.Lerp(horizontal, Vector3.zero, stopDamping * Time.fixedDeltaTime);
    _rb.linearVelocity = new Vector3(damped.x, vel.y, damped.z);
}

private void LimitHorizontalSpeed()
{
    Vector3 vel = _rb.linearVelocity;
    Vector3 horizontal = new Vector3(vel.x, 0f, vel.z);

    if (horizontal.magnitude > maxSpeed)
    {
        Vector3 limited = horizontal.normalized * maxSpeed;
        _rb.linearVelocity = new Vector3(limited.x, vel.y, limited.z);
    }
}
}
// ===== END SECTION =====

```

Tennis - CollectableManager.cs

Provenance header

- Project: Tennis
- File: CollectableManager.cs
- Authorship status: Written by me without assistance.

Code (text format):

```
// ===== BEGIN SECTION =====
using UnityEngine;
using TMPro;

public class CollectableManager : MonoBehaviour
{
    public static CollectableManager Instance { get; private set; }

    [Header("Progress")]
    public int requiredToUnlock = 3; // set this to your number of collectables
    public int collected = 0;

    [Header("Gate")]
    public GateController gateToUnlock;

    [Header("UI (optional)")]
    public TextMeshProUGUI counterText;

    private void Awake()
    {
        if (Instance != null && Instance != this)
        {
            Destroy(gameObject);
            return;
        }
        Instance = this;
    }

    private void Start()
    {
        // Always reset on Play
        collected = 0;
        UpdateUI();

        // Ensure gate starts visible/solid every Play
        if (gateToUnlock != null)
            gateToUnlock.ResetGate();
    }

    public void AddOne()
    {
        collected++;
        UpdateUI();

        if (gateToUnlock != null && collected >= requiredToUnlock)
            gateToUnlock.UnlockGate();
    }

    private void UpdateUI()
    {
        if (counterText != null)
```

```
        counterText.text = $"Collected: {collected}/{requiredToUnlock}";
    }
}
// ===== END SECTION =====
```

Tennis - CollectablePickup.cs

Provenance header

- Project: Tennis
- File: CollectablePickup.cs
- Authorship status: Written by me without assistance.

Code (text format):

```
// ===== BEGIN SECTION =====
using UnityEngine;

public class CollectablePickup : MonoBehaviour
{
    private bool _pickedUp;

    private void OnTriggerEnter(Collider other)
    {
        if (_pickedUp) return;
        if (!other.CompareTag("Player")) return;

        _pickedUp = true;

        if (CollectableManager.Instance != null)
        {
            CollectableManager.Instance.AddOne();
        }
        else
        {
            Debug.LogError("CollectableManager.Instance is null. Make sure a GameManager with CollectableManager exists in the scene.");
        }

        Destroy(gameObject);
    }
}
// ===== END SECTION =====
```

Tennis - GateController.cs

Provenance header

- Project: Tennis
- File: GateController.cs
- Authorship status: Written by me without assistance.

Code (text format):

```
// ===== BEGIN SECTION =====
using UnityEngine;

public class GateController : MonoBehaviour
{
    [Header("Disappear behaviour")]
    [Tooltip("If true, the script auto-fills the arrays from this object and its children.")]
    public bool autoFindComponents = true;

    [Tooltip("All colliders to disable on unlock (door + lock + any child colliders).")]
    public Collider[] collidersToDisable;

    [Tooltip("All renderers to disable on unlock (door visuals + any child visuals).")]
    public Renderer[] renderersToDisable;

    [Header("Optional: keep door frozen before unlock")]
    public Rigidbody gateRb;
    public bool freezeOnStart = true;

    private bool _unlocked;

    private void Awake()
    {
        if (gateRb == null) gateRb = GetComponent<Rigidbody>();

        if (autoFindComponents)
        {
            // Auto-collect everything on this object and children
            collidersToDisable = GetComponentsInChildren<Collider>(includeInactive: true);
            renderersToDisable = GetComponentsInChildren<Renderer>(includeInactive: true);
        }

        ResetGate();
    }

    public void ResetGate()
    {
        _unlocked = false;

        // Re-enable collision
        if (collidersToDisable != null)
        {
            foreach (var c in collidersToDisable)
            {
                if (c != null) c.enabled = true;
            }
        }

        // Re-enable visuals
        if (renderersToDisable != null)
        {
            foreach (var r in renderersToDisable)
            {
                if (r != null) r.enabled = true;
            }
        }
    }
}
```

```

    }

    // Optional: stop physics drift
    if (freezeOnStart && gateRb != null)
    {
        gateRb.useGravity = false;
        gateRb.linearVelocity = Vector3.zero;
        gateRb.angularVelocity = Vector3.zero;
        gateRb.constraints = RigidbodyConstraints.FreezeAll;
    }
}

public void UnlockGate()
{
    if (_unlocked) return;
    _unlocked = true;

    Debug.Log("Gate unlocked -> door disappearing");

    // Disable colliders first so it stops blocking
    if (collidersToDisable != null)
    {
        foreach (var c in collidersToDisable)
            if (c != null) c.enabled = false;
    }

    // Disable renderers so it becomes invisible
    if (renderersToDisable != null)
    {
        foreach (var r in renderersToDisable)
            if (r != null) r.enabled = false;
    }

    // Optional: keep rigidbody frozen after unlock too
    if (gateRb != null)
    {
        gateRb.linearVelocity = Vector3.zero;
        gateRb.angularVelocity = Vector3.zero;
        gateRb.constraints = RigidbodyConstraints.FreezeAll;
    }
}
}

// ===== END SECTION =====

```

Tennis - ThirdPersonOrbitCamera.cs

Provenance header

- Project: Tennis
- File: ThirdPersonOrbitCamera.cs
- Authorship status: Written by me without assistance.

Code (text format):

```
// ===== BEGIN SECTION =====
using UnityEngine;

public class ThirdPersonOrbitCamera : MonoBehaviour
{
    [Header("Target")]
    public Transform target;

    [Header("Orbit")]
    public float distance = 6f;
    public float heightOffset = 1.0f;
    public float mouseSensitivity = 2.0f;

    [Tooltip("Up/down look limits.")]
    public float minPitch = -20f;
    public float maxPitch = 70f;

    [Header("Smoothing")]
    public float positionSmoothTime = 0.08f;

    [Header("Input")]
    public bool holdRightMouseToRotate = false;

    [Header("Collision (optional)")]
    public bool collisionEnabled = true;
    public float collisionRadius = 0.2f;
    public float collisionBuffer = 0.1f;
    public LayerMask collisionMask = ~0; // everything

    private float _yaw;
    private float _pitch;
    private Vector3 _posVel;

    private Camera _cam;

    private void Awake()
    {
        _cam = GetComponentInChildren<Camera>();
        if (_cam == null) _cam = Camera.main;

        // Start from current view direction
        Vector3 euler = transform.eulerAngles;
        _yaw = euler.y;
        _pitch = euler.x;
    }

    private void Start()
    {
        // Optional: lock cursor for a typical third-person feel
        Cursor.lockState = CursorLockMode.Locked;
        Cursor.visible = false;
    }
}
```

```

}

private void LateUpdate()
{
    if (target == null || _cam == null) return;

    // 1) Mouse input
    bool allowRotate = !holdRightMouseToRotate || Input.GetMouseButton(1);

    if (allowRotate)
    {
        float mx = Input.GetAxis("Mouse X");
        float my = Input.GetAxis("Mouse Y");

        _yaw += mx * mouseSensitivity;
        _pitch -= my * mouseSensitivity;
        _pitch = Mathf.Clamp(_pitch, minPitch, maxPitch);
    }

    // 2) Desired rotation from yaw/pitch
    Quaternion rot = Quaternion.Euler(_pitch, _yaw, 0f);

    // 3) Desired pivot point (target + height)
    Vector3 pivot = target.position + Vector3.up * heightOffset;

    // 4) Desired camera position (behind pivot)
    Vector3 desiredCamPos = pivot - (rot * Vector3.forward * distance);

    // 5) Collision (sphere cast from pivot to desired camera position)
    if (collisionEnabled)
    {
        Vector3 dir = (desiredCamPos - pivot);
        float desiredDist = dir.magnitude;
        if (desiredDist > 0.001f)
        {
            dir /= desiredDist;

            if (Physics.SphereCast(pivot, collisionRadius, dir, out RaycastHit hit, desiredDist, collisionMask))
            {
                float safeDist = Mathf.Max(0.5f, hit.distance - collisionBuffer);
                desiredCamPos = pivot + dir * safeDist;
            }
        }
    }

    // 6) Smooth camera movement (rig position)
    transform.position = Vector3.SmoothDamp(transform.position, desiredCamPos, ref _posVel, positionSmoothTime);

    // 7) Look at target pivot
    transform.rotation = rot;
    _cam.transform.LookAt(pivot);
}
}

// ===== END SECTION =====

```

Tennis - ReadmeEditor.cs

Provenance header

- Project: Tennis
- File: ReadmeEditor.cs
- Authorship status: EXTERNAL TEMPLATE: Included from Unity project template or sample content.

Code (text format):

```
// ===== BEGIN SECTION =====
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEditor;
using System;
using System.IO;
using System.Reflection;

[CustomEditor(typeof(Readme))]
[InitializeOnLoad]
public class ReadmeEditor : Editor
{
    static string s_ShowedReadmeSessionStateName = "ReadmeEditor.showedReadme";

    static string s_ReadmeSourceDirectory = "Assets/TutorialInfo";

    const float k_Space = 16f;

    static ReadmeEditor()
    {
        EditorApplication.delayCall += SelectReadmeAutomatically;
    }

    static void RemoveTutorial()
    {
        if (EditorUtility.DisplayDialog("Remove Readme Assets",

            $"All contents under {s_ReadmeSourceDirectory} will be removed, are you sure you want to proceed?",
            "Proceed",
            "Cancel"))
        {
            if (Directory.Exists(s_ReadmeSourceDirectory))
            {
                FileUtil.DeleteFileOrDirectory(s_ReadmeSourceDirectory);
                FileUtil.DeleteFileOrDirectory(s_ReadmeSourceDirectory + ".meta");
            }
            else
            {
                Debug.Log($"Could not find the Readme folder at {s_ReadmeSourceDirectory}");
            }

            var readmeAsset = SelectReadme();
            if (readmeAsset != null)
            {
                var path = AssetDatabase.GetAssetPath(readmeAsset);
                FileUtil.DeleteFileOrDirectory(path + ".meta");
                FileUtil.DeleteFileOrDirectory(path);
            }

            AssetDatabase.Refresh();
        }
    }
}
```

```

    }
}

static void SelectReadmeAutomatically()
{
    if (!SessionState.GetBool(s_ShowedReadmeSessionStateName, false))
    {
        var readme = SelectReadme();
        SessionState.SetBool(s_ShowedReadmeSessionStateName, true);

        if (readme && !readme.loadedLayout)
        {
            LoadLayout();
            readme.loadedLayout = true;
        }
    }
}

static void LoadLayout()
{
    var assembly = typeof(EditorApplication).Assembly;
    var windowLayoutType = assembly.GetType("UnityEditor.WindowLayout", true);
    var method = windowLayoutType.GetMethod("LoadWindowLayout", BindingFlags.Public | BindingFlags.Static);
    method.Invoke(null, new object[] { Path.Combine(Application.dataPath, "TutorialInfo/Layout.wlt"), false });
}

static Readme SelectReadme()
{
    var ids = AssetDatabase.FindAssets("Readme t:Readme");
    if (ids.Length == 1)
    {
        var readmeObject = AssetDatabase.LoadMainAssetAtPath(AssetDatabase.GUIDToAssetPath(ids[0]));

        Selection.objects = new UnityEngine.Object[] { readmeObject };

        return (Readme)readmeObject;
    }
    else
    {
        Debug.Log("Couldn't find a readme");
        return null;
    }
}

protected override void OnHeaderGUI()
{
    var readme = (Readme)target;
    Init();

    var iconWidth = Mathf.Min(EditorGUIUtility.currentViewWidth / 3f - 20f, 128f);

    GUILayout.BeginHorizontal("In BigTitle");
    {
        if (readme.icon != null)
        {
            GUILayout.Space(k_Space);
            GUILayout.Label(readme.icon, GUILayout.Width(iconWidth), GUILayout.Height(iconWidth));
        }
        GUILayout.Space(k_Space);
        GUILayout.BeginVertical();
        {
            GUILayout.FlexibleSpace();
            GUILayout.Label(readme.title, TitleStyle);
            GUILayout.FlexibleSpace();
        }
        GUILayout.EndVertical();
        GUILayout.FlexibleSpace();
    }
}

```

```

    }
    GUILayout.EndHorizontal();
}

public override void OnInspectorGUI()
{
    var readme = (Readme)target;
    Init();

    foreach (var section in readme.sections)
    {
        if (!string.IsNullOrEmpty(section.heading))
        {
            GUILayout.Label(section.heading, HeadingStyle);
        }

        if (!string.IsNullOrEmpty(section.text))
        {
            GUILayout.Label(section.text, BodyStyle);
        }

        if (!string.IsNullOrEmpty(section.linkText))
        {
            if (LinkLabel(new GUIContent(section.linkText)))
            {
                Application.OpenURL(section.url);
            }
        }

        GUILayout.Space(k_Space);
    }

    if (GUILayout.Button("Remove Readme Assets", ButtonStyle))
    {
        RemoveTutorial();
    }
}

bool m_Initialized;

GUIStyle LinkStyle
{
    get { return m_LinkStyle; }
}

[SerializeField]
GUIStyle m_LinkStyle;

GUIStyle TitleStyle
{
    get { return m_TitleStyle; }
}

[SerializeField]
GUIStyle m_TitleStyle;

GUIStyle HeadingStyle
{
    get { return m_HeadingStyle; }
}

[SerializeField]
GUIStyle m_HeadingStyle;

GUIStyle BodyStyle
{
    get { return m_BodyStyle; }
}

```

```

[SerializeField]
GUIStyle m_BodyStyle;

GUIStyle ButtonStyle
{
    get { return m_ButtonStyle; }
}

[SerializeField]
GUIStyle m_ButtonStyle;

void Init()
{
    if (m_Initialized)
        return;
    m_BodyStyle = new GUIStyle(EditorStyles.label);
    m_BodyStyle.wordWrap = true;
    m_BodyStyle.fontSize = 14;
    m_BodyStyle.richText = true;

    m_TitleStyle = new GUIStyle(m_BodyStyle);
    m_TitleStyle.fontSize = 26;

    m_HeadingStyle = new GUIStyle(m_BodyStyle);
    m_HeadingStyle.fontStyle = FontStyle.Bold;
    m_HeadingStyle.fontSize = 18;

    m_LinkStyle = new GUIStyle(m_BodyStyle);
    m_LinkStyle.wordWrap = false;

    // Match selection color which works nicely for both light and dark skins
    m_LinkStyle.normal.textColor = new Color(0x00 / 255f, 0x78 / 255f, 0xDA / 255f, 1f);
    m_LinkStyle.stretchWidth = false;

    m_ButtonStyle = new GUIStyle(EditorStyles.miniButton);
    m_ButtonStyle.fontStyle = FontStyle.Bold;

    m_Initialized = true;
}

bool LinkLabel(GUIContent label, params GUILayoutOption[] options)
{
    var position = GUILayoutUtility.GetRect(label, LinkStyle, options);

    Handles.BeginGUI();
    Handles.color = LinkStyle.normal.textColor;
    Handles.DrawLine(new Vector3(position.xMin, position.yMax), new Vector3(position.xMax, position.yMax));
    Handles.color = Color.white;
    Handles.EndGUI();

    EditorGUIUtility.AddCursorRect(position, MouseCursor.Link);

    return GUI.Button(position, label, LinkStyle);
}
}
// ===== END SECTION =====

```

Tennis - Readme.cs

Provenance header

- Project: Tennis
- File: Readme.cs
- Authorship status: EXTERNAL TEMPLATE: Included from Unity project template or sample content.

Code (text format):

```
// ===== BEGIN SECTION =====
using System;
using UnityEngine;

public class Readme : ScriptableObject
{
    public Texture2D icon;
    public string title;
    public Section[] sections;
    public bool loadedLayout;

    [Serializable]
    public class Section
    {
        public string heading, text, linkText, url;
    }
}
// ===== END SECTION =====
```